

8.2 Tensor Decomposition Algorithms

CP, Tucker, and how the factors are computed.

These notes walk through the lecture slides. The original deck is unchanged; use **(slides)** on the course hub if you want that PDF.

Figures and notes follow *Introduction to Tensor Decompositions and their Applications in Machine Learning*.

1. Tucker decomposition

Tucker writes a tensor as a **core tensor** plus a factor matrix per mode. It is a higher-order PCA.

Core tensor. A compressed version of the original: the essential interactions, in a smaller array.

Factor matrices. Principal components in that mode. Multiply them with the core to stretch back to the original sizes.

2. Why higher-order PCA?

PCA decomposes a **matrix** into orthogonal components that capture variance. Tucker does the same for a **tensor**: a core, and matrices that capture variance along each **mode**.

PCA reduces matrix dimension by keeping the important components. Tucker reduces tensor dimension by keeping the important **interactions**. That is the analysis and visualization move for multiway data.

3. Practical benefits

Compression. Choose a core smaller than $\mathcal{X}$. Less storage, cheaper arithmetic.

Interpretability. Each column of a factor matrix is a principal component along that mode.

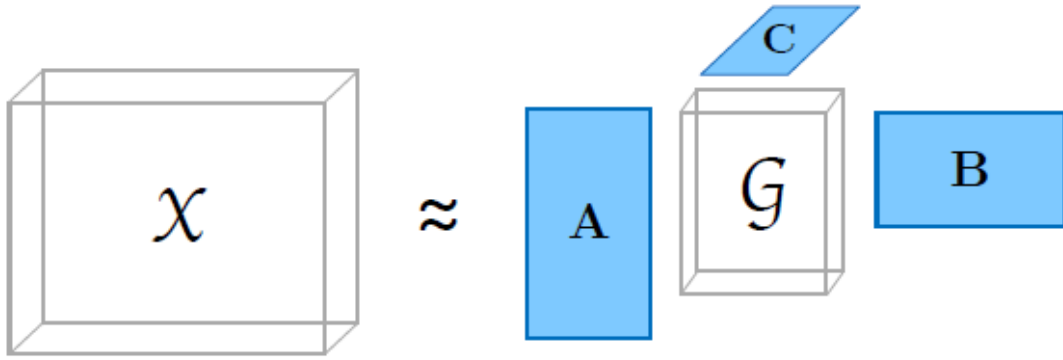
Flexibility. You choose the core size, so you choose how much you compress versus how much detail you keep. Other decompositions are less free here.

Picture the core as a small dense block of interactions, and each factor matrix as axes that stretch or compress that block back toward the data.

4. Problem formulation

For $\mathcal{X} \in \mathbb{R}^{I \times J \times K}$, find $\mathcal{G} \in \mathbb{R}^{P \times Q \times R}$, $\mathbf{A} \in \mathbb{R}^{I \times P}$, $\mathbf{B} \in \mathbb{R}^{J \times Q}$, $\mathbf{C} \in \mathbb{R}^{K \times R}$ solving

$$\min_{\hat{\mathcal{X}}} \|\mathcal{X} - \hat{\mathcal{X}}\| \quad \text{with} \quad \hat{\mathcal{X}} = \sum_{p=1}^P \sum_{q=1}^Q \sum_{r=1}^R g_{pqr} \mathbf{a}_p \odot \mathbf{b}_q \odot \mathbf{c}_r = \mathcal{G} \times_1 \mathbf{A} \times_2 \mathbf{B} \times_3 \mathbf{C}.$$



Tucker Decomposition

- $\mathcal{G}$ is the core: how, and how much, elements interact.
- $\mathbf{A}, \mathbf{B}, \mathbf{C}$ are the principal components in each mode.
- If $P < I, Q < J, R < K$, you compress: $\mathcal{G}$ is the compressed $\mathcal{X}$.

Matricized form ($\otimes$ is Kronecker):

$$\hat{\mathcal{X}}_{(1)} = \mathbf{A} G_{(1)} (\mathbf{C} \otimes \mathbf{B})^\top,$$

$$\hat{\mathcal{X}}_{(2)} = \mathbf{B} G_{(1)} (\mathbf{C} \otimes \mathbf{A})^\top,$$

$$\hat{\mathcal{X}}_{(3)} = \mathbf{C} G_{(1)} (\mathbf{B} \otimes \mathbf{A})^\top.$$

N-way:

$$\hat{\mathcal{X}} = \sum_{r_1=1}^{R_1} \cdots \sum_{r_N=1}^{R_N} g_{r_1 \dots r_N} \mathbf{a}_{i_1 r_1}^{(1)} \odot \cdots \odot \mathbf{a}_{i_N r_N}^{(N)} = \mathcal{G} \times_1 \mathbf{A}^{(1)} \times_2 \cdots \times_N \mathbf{A}^{(N)},$$

$$\hat{\mathcal{X}}_{(n)} = \mathbf{A}^{(n)} G_{(n)} (\mathbf{A}^{(1)} \otimes \cdots \otimes \mathbf{A}^{(n+1)} \otimes \mathbf{A}^{(n-1)} \otimes \cdots \otimes \mathbf{A}^{(1)})^\top.$$

5. n-rank

Before computing Tucker, you need **n-rank**.

The n -rank of $\mathcal{X} \in \mathbb{R}^{I_1 \times \cdots \times I_N}$ is the **column rank** of the n th unfolding $X_{(n)}$, written $\text{rank}_n(\mathcal{X})$.

This is **not** the tensor rank of the previous note.

6. Higher-order SVD (HOSVD)

For a given $\mathcal{X}$ there is an **exact** Tucker decomposition of rank $(R_1, \dots, R_N)$ with $R_n = \text{rank}_n(\mathcal{X})$. That factorization is **HOSVD**.

The idea: in each mode n , find the components that best capture variation in that mode **alone**, ignoring the others for that step. PCA, one unfolding at a time.

HOSVD $(\mathcal{X}; R_1, \dots, R_N)$:

1. For $n = 1, \dots, N$: set $\mathbf{A}^{(n)}$ to the R_n leading left singular vectors of $X_{(n)}$.
2. Core:

$$\mathcal{G} = \mathcal{X} \times_1 \mathbf{A}^{(1)\top} \times_2 \dots \times_N \mathbf{A}^{(N)\top}.$$

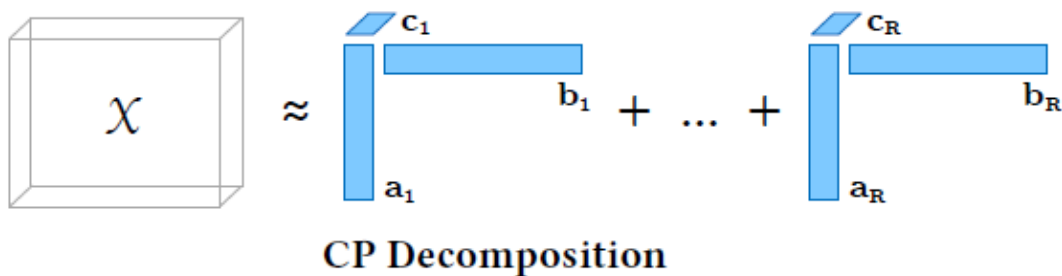
3. Return $\mathcal{G}, \mathbf{A}^{(1)}, \dots, \mathbf{A}^{(N)}$.

7. Limitations of Tucker

- **Cost.** Factor matrices plus core are expensive for large tensors and high ranks.
- **Memory.** The core and the factors can be large.
- **Initialization.** Quality can depend on the starting factors; optimality is not guaranteed.

8. Canonical polyadic decomposition (CPD)

An alternative: write the tensor as a sum of a finite number of **rank-1** tensors. CPD comes from **CANDECOMP** (canonical decomposition) and **PARAFAC** (parallel factors).



	Tucker	CPD
Flexibility	Different ranks per mode; complex interactions	Rank-1 sum; simpler structure
Interpretability	Core shows how modes interact	Factor matrices as patterns in each mode
Compression / scale	Can compress well when interactions are complex	Cheaper; often scales better on large or high-order data

Practice

1. In Tucker, what does the **core tensor** store that a CPD sum of rank-1 terms does not?
2. How does **HOSVD** choose each factor matrix $\mathbf{A}^{(n)}$?
3. When might you prefer **CPD** to Tucker?